Transformação de Perspectiva por 4 Pontos Selecionados pelo Usuário

TA03 — Visão Computacional

Outubro de 2026

Resumo

Este artigo descreve um modelo interativo de correção de perspectiva em que o usuário seleciona 4 pontos na imagem para delimitar o objeto de interesse. A partir desses pontos, o programa estima a transformação que leva o quadrilátero selecionado a uma vista frontal retangular de tamanho fixo, usando OpenCV. O experimento foi realizado sobre uma imagem de Sudoku, e são apresentados a tela de seleção, a imagem de saída e o comparativo entrada/saída.

Palavras-chave: perspectiva, seleção interativa, homografia, OpenCV.

1 Introdução

A transformação de perspectiva corrige a distorção projetiva de objetos planares fotografados de ângulo ou com bordas extras, gerando uma vista frontal. É a base de digitalização de documentos, leitura de placas e tabuleiros e pré-processamento para OCR.

O diferencial deste trabalho é o **modelo do problema ser interativo**: em vez de detectar os cantos automaticamente, é o usuário quem delimita a região. Isso torna o programa genérico — funciona para qualquer objeto quadrangular — ao custo de exigir 4 cliques precisos. O programa (`main.py`) recebe a imagem, abre a janela de seleção, calcula a transformação e salva a vista frontal em `saida/`, exibindo o comparativo lado a lado.

2 Metodologia

2.1 Modelo do problema: o usuário delimita a região

O problema é modelado como: dada uma imagem de entrada qualquer, o usuário indica com o mouse os 4 cantos do objeto (no experimento, o grid 9×9 do Sudoku). Cada clique registra uma coordenada (x, y) e desenha um marcador visual (círculo vermelho + número verde) para feedback imediato.

A interface foi pensada para ser tolerante a erros: a tecla `r` reinicia a seleção, `q` ou ESC cancela, e fechar a janela também cancela sem efeitos colaterais. Só após o quarto ponto a transformação é calculada, com uma pequena pausa para o usuário conferir o último marcador. A Figura 1 mostra essa etapa.

Por trás, um *callback* de mouse do OpenCV (`setMouseCallback` com evento `EVENT_LBUTTONDOWN`) acumula os pontos em uma lista até completar 4, sem travar a interface (laço com `waitKey`).

2.2 Ordenação: aceitar cliques em qualquer ordem

Exigir uma ordem fixa de clique (ex. horário) seria propenso a erro. Por isso há uma etapa de ordenação automática: a partir das coordenadas, o programa identifica qual ponto é o topo-esquerdo, topo-direito, base-esquerda e base-direita, usando apenas somas e diferenças de x e y. Assim o usuário pode clicar nos cantos em qualquer sequência que o destino canônico — os quatro cantos de um retângulo de tamanho $w \times h$ (padrão 300×300, configurável por linha de comando) — receba as correspondências corretas.

2.3 Estimativa da transformação e geração da saída

Com os 4 pares origem $\leftrightarrow$ destino definidos, o OpenCV estima a matriz de transformação de perspectiva (homografia 3×3) pela função `getPerspectiveTransform`. Intuitivamente, ela descreve como "esticar" o quadrilátero irregular da foto até virar um retângulo perfeito; 4 pontos são o mínimo necessário para determinar essa deformação.

Em seguida, `warpPerspective` percorre cada pixel da imagem de destino e busca o pixel correspondente na origem com interpolação, evitando buracos. O resultado é exibido com matplotlib (Input × Output) e salvo com nome versionado (`saida/<nome>_perspectiva_<n>.jpg`) para não sobrescrever execuções anteriores:

```
python3 main.py entrada/sudoku.jpg --largura 300 --altura 300
```

3 Resultados

Entrada. A imagem `entrada/sudoku.jpg` contém o tabuleiro com texto abaixo ("Hoffman's Difficult Sudoku"). O objetivo é isolar só o grid.

Seleção. A Figura 1 mostra a janela no momento da seleção: os marcadores 1, 2 e 3 visíveis nos cantos do tabuleiro confirmam os cliques. Essa etapa é rápida (segundos) e dispensa qualquer calibração.

Saída e comparativo. A Figura 2 reúne entrada, saída retificada e o comparativo automático. O grid passa a ocupar todo o quadro 300×300, frontal e sem o texto — vista normalizada pronta para detecção de células ou OCR.

Observou-se que a qualidade depende diretamente da precisão do clique: seleções descuidadas geram zoom excessivo ou cortes nas bordas (como em `sudoku_perspectiva_1.jpg` e `_2.jpg`). Com cliques nos cantos exatos (`_3.jpg`, usada na figura), o resultado é limpo e estável entre execuções.

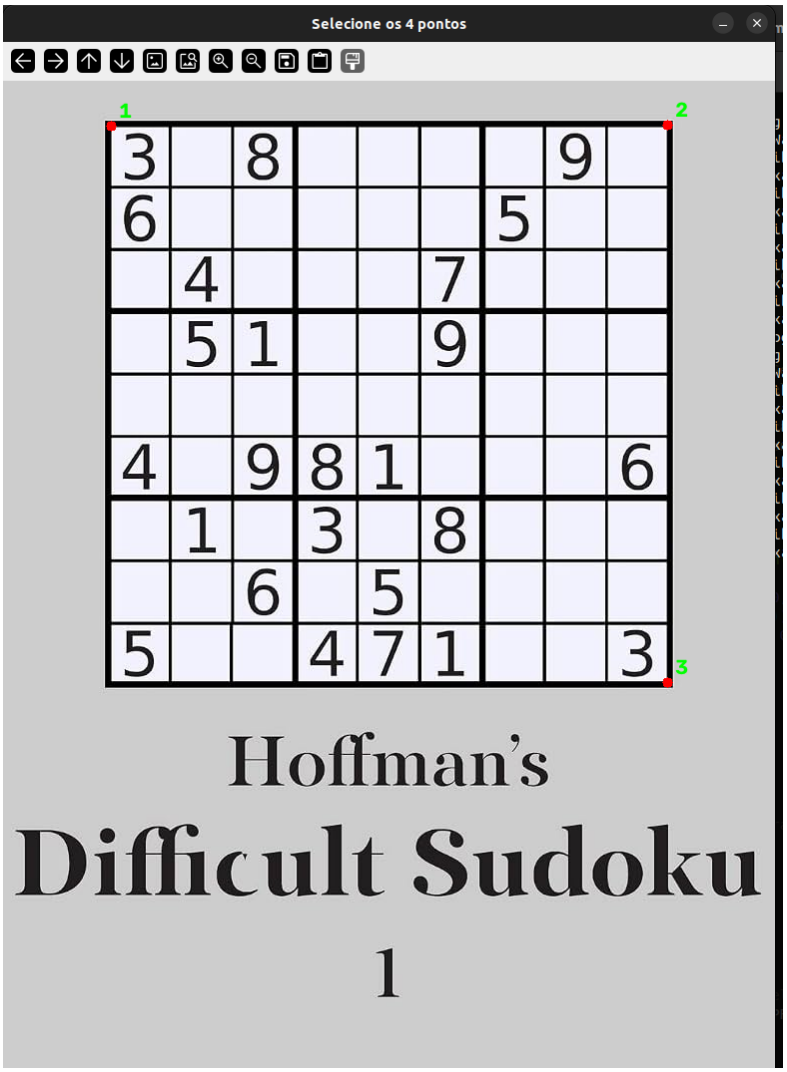
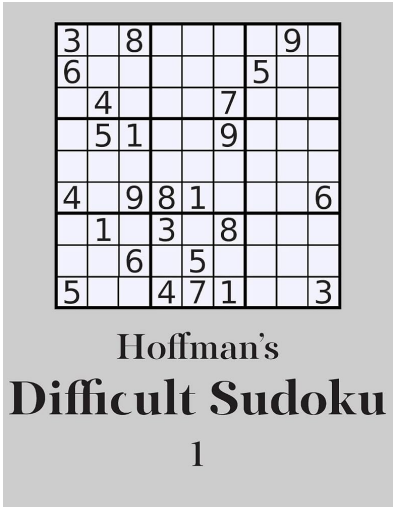
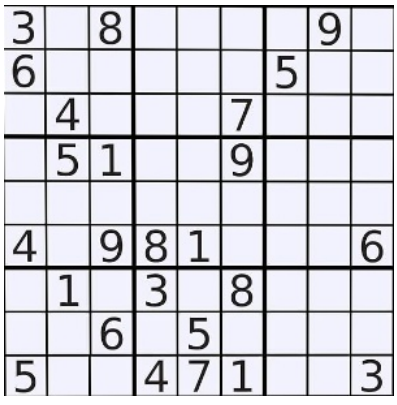


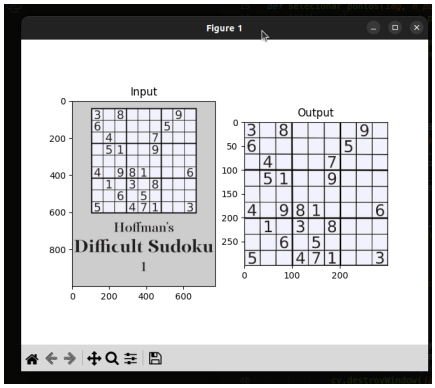
Figura 1: Tela de seleção: o usuário clica nos 4 cantos do Sudoku. `r` reinicia, `q`/ESC cancela.



Entrada



Saída retificada



Comparativo Input × Output

Figura 2: Entrada, saída e comparativo gerado pelo programa.

4 Conclusão

O modelo interativo mostrou-se simples e eficaz: 4 cliques delimitam qualquer região quadrangular e produzem uma vista frontal adequada. A principal limitação é a sensibilidade à precisão manual e o tamanho fixo de saída. Como evolução, sugere-se detecção automática dos cantos e integração com OCR das células.

Referências

[1] Documentação OpenCV, `https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html`.

[2] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2022.